

Programowanie współbieżne w C++

Lista zadań nr 13

Na pracownię 11 czerwca 2026

Przed rozpoczęciem rozwiązywania Zadań 1 i 2, przeczytaj **poprawioną** notatkę dotyczącą zastosowania Epoch-Based Reclamation i Interval-Based Reclamation zamieszczoną na stronie wykładu.

Zadanie 1 (5pktów). Epoch-Based Reclamation w praktyce. Ponownie.

Wybierz dowolne zadanie z poprzednich list, w którym problemem była bezpieczna dealokacja pamięci. Zaimplementuj w nim mechanizm EBR.

Zadanie 2 (7pktów). Interval-Based Reclamation w praktyce. Ponownie.

Wybierz dowolne zadanie z poprzednich list, w którym problemem była bezpieczna dealokacja pamięci. Zaimplementuj w nim mechanizm IBR.

Poniższe zadania dotyczą ogólnego podejścia do projektowania współbieżnych struktur danych. W rozwiązaniach użyj blokad, ale postaraj się zrobić to efektywnie!

Zadanie 3 (7pktów). Wielowątkowy HistorySet<T> z blokadą.

Zaimplementuj szablon klasy `HistorySet<T>`, będącej zbiorem z funkcjami `insert`, `erase` oraz `contains`, a także z funkcją `undo`, cofającą ostatnio wykonaną operację. Funkcja `undo` powinna być wielopoziomowa – należy jawnie zapamiętać historię wszystkich operacji. Jako bazowe struktury danych wykorzystaj dowolne kontenery z biblioteki standardowej. Zastanów się, jakie dodatkowe operatory na typie `T` mogą powodować wyjątki.

Zadanie 4 (8pktów). Wielowątkowy zbiór z blokadą i migawkami.

Zaimplementuj szablon klasy `VersionedConcurrentSet<T>`, która umożliwia wielu wątkom równoległą pracę (`insert`, `erase`, `find`)

na własnych kopiach (migawkach) zbioru. Główny zbiór jest aktualizowany tylko podczas rzadkiego, zsynchronizowanego scalania (**merge**) zmian z migawek. Dzięki temu większość operacji nie wymaga blokad, a synchronizacja jest ograniczona do momentów scalania. Zadbaj o poprawną implementację operacji scalenia, zgodnie z zasadą "ostatni zapis wygrywa". Oznacza to w szczególności, że scalenie polegające na zsumowaniu wszystkich zbiorów-migawek jest niepoprawne. Być może przydadzą się znaczniki czasu.